UNIVERSIDADE ESTADUAL DO CEARÁ

	AV1	AV2	AF
1ª ch	x		
2ª ch			

Curso: Ciência da Computação | Disciplina: Estrutura de Dados

Professor/a: Bonfim Amaro Júnior | Data: 08/04/2026

Aluno/a: | Matrícula:

Observações:

1. Avaliação individual e sem consulta;
2. Tempo de duração da avaliação: 1h40m;
3. Responder de forma organizada e com letra legível;
4. Circule as questões respondidas;

Conforme estabelecido nos critérios de pontuação, esta prova vale 10,0 (Dez) pontos.

GABARITO / ESPELHO DA PROVA

01. Escreva a função *filtrar* em C que possui a assinatura e os requisitos abaixo **(2,0 pontos)**:

```
int* filtrar (int *v, int n, int corte, int *qtd);
```

- ✓ receba como parâmetros:
  - um vetor de inteiros v;
  - o tamanho do vetor n;
  - um inteiro corte;
  - um ponteiro para inteiro qtd, que deverá armazenar a quantidade de elementos do vetor resultante;
- ✓ retorne um **novo vetor alocado dinamicamente** contendo **apenas os elementos de v que são maiores ou iguais a corte**;
- ✓ atualize o valor apontado por qtd com a quantidade de elementos do novo vetor.

Se	Então
v = {2, 7, 3, 10, 5}	Vetor resultado = {7, 10, 5}
corte = 5	*qtd = 3

Critérios :

- ✓ contagem correta dos elementos >= corte: **0,40**
- ✓ uso correto de *qtd: **0,30**
- ✓ alocação dinâmica correta: **0,40**
- ✓ preenchimento correto do vetor resultante: **0,50**
- ✓ tratamento / robustez: **0,40**

RESPOSTA:

```
#include <stdlib.h>

int* filtrar(int *v, int n, int corte, int *qtd) {
    int cont = 0;
```

```
    /* 1ª passagem: contar quantos elementos satisfazem a condição */
    for (int i = 0; i < n; i++) {
        if (v[i] >= corte) {
            cont++;
        }
    }

    *qtd = cont;

    /* se não houver elementos, pode retornar NULL */
    if (cont == 0) {
        return NULL;
    }

    /* 2ª passagem: alocar e preencher o novo vetor */
    int *novo = (int*) malloc(cont * sizeof(int));
    if (novo == NULL) {
        *qtd = 0;
        return NULL;
    }

    int j = 0;
    for (int i = 0; i < n; i++) {
        if (v[i] >= corte) {
            novo[j] = v[i];
            j++;
        }
    }

    return novo;
}
```

02. Considere a estrutura abaixo, usada para representar um aluno **(1,5 ponto)**:

```
struct Aluno {
    char nome[50];
    int idade;
    float nota;
};
```

Complete corretamente o código da função criarAluno, preenchendo as lacunas com comandos em C, de modo que:

- seja alocada dinamicamente memória para um aluno;
- os campos da estrutura sejam acessados corretamente;
- os valores de idade e nota sejam armazenados na estrutura;
- a função retorne o ponteiro para a estrutura criada;
- caso a alocação falhe, a função retorne NULL.

```
#include <stdlib.h>

struct Aluno {
char nome[50];
int idade;
```

```
float nota;
};

struct Aluno* criarAluno(int idade, float nota) {
struct Aluno *a = ____________________________________; 1

if (a == NULL) {
return ____________________________________; 2

}

____________________________________ = idade; 3
____________________________________ = nota; 3

return ____________________________________; 4

}
```

Critérios:

- ✓ 1. alocação dinâmica correta: 0,70
- ✓ 2. retorno correto de tratamento: 0,20
- ✓ 3. acesso correto aos campos: 0,30
- ✓ 4. retorno correto: 0,30

RESPOSTA:

```
1 . (struct Aluno*) malloc(sizeof(struct Aluno)) ou
    (struct Aluno*) malloc(sizeof(struct Aluno))
2. NULL
3. a->idade ou (*a).idade | a->idade ou (*a).idade
4. a
```

03. Considere o código em C **(1,5 ponto)**:

```
#include <stdio.h>

int main() {
    int v[] = {10, 20, 30, 40, 50};
    int *p = v;

    int a = *(p + 2);
    int b = *(v + 4);
    int c = *(p++);
    int d = *p;

    printf("%d %d %d %d\n", a, b, c, d);

    return 0;
}
```

Apresente a saída do printf, ou seja, a saída do programa:

Critérios:

- ✓ Acertar o valor das variáveis a e b: 0,25 cada
- ✓ Acertar o valor das variáveis c e d: 0,50 cada

RESPOSTA:

a = *(p + 2) | p aponta para v[0], então p+2 aponta para v[2]
a = 30

b = *(v + 4) | v+4 aponta para v[4]
b = 50

c = *(p++) | usa primeiro o valor atual de p, depois incrementa então pega v[0] = 10, e depois p passa a apontar para v[1]
c = 10

d = *p | agora p aponta para v[1]
d = 20

SAÍDA: 30 50 10 20

04. Considere as funções f1 e f2 a seguir:

```
int f1(int n) {
    int s = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            s++;
        }
    }

       Continua na 2ª coluna....
```

```
    for (int k = 1; k < n; k = k * 2) {
        for (int t = 0; t < n; t++) {
                s++;
        }
    }

    return s;
}
```

```
int f2(int *v, int n) {
    if (n <= 0) {
        return 0;
    }

    int soma = 0;
    for (int i = 0; i < n; i++) {
        soma += v[i];
    }

    return soma + f2(v, n - 1);
}
```

Responda, **justificando sua resposta**:

a) Qual é a complexidade assintótica do **segundo** bloco de repetição da **função f1**? (1,0 ponto)

b) Qual é a complexidade assintótica total **da função f1**? (1,0 ponto)

c) Qual a saída **da função f2** para essa chamada na main ? (1,0 ponto)

```
int main() {
```

```
    int v[] = {1,2,3,4};
    int n = sizeof(v)/sizeof(v[0]);
    printf("%d\n", f2(v, n));

}
```

d) Qual a complexidade assintótica total da **função f2**? (1,5 ponto)

RESPOSTA:

a) O laço externo, controlado por k, dobra o valor a cada iteração (1, 2, 4, 8, ...), portanto executa aproximadamente $\log_2(n)$ vezes.

Já o laço interno executa n vezes para cada iteração do laço externo.
Assim, a quantidade total de operações desse bloco é: n*log(n)
Logo, a complexidade assintótica do segundo bloco é: **Θ(n log n)**

b) O laço externo executa n vezes e, para cada execução, o laço interno também executa n vezes.
Portanto, esse bloco possui complexidade:
Θ(n²)

Segundo bloco:
Já analisado anteriormente, possui complexidade:
Θ(n log n)

Soma das complexidades:
Θ(n²) + Θ(n log n)

Como o termo dominante é n², a complexidade assintótica total da função f1 é: **Θ(n²)**

c)

f2(v,4) = 1 + 2 + 3 + 4 = 10
f2(v,3) = 1 + 2 + 3 = 6
f2(v,2) = 1 + 2 = 3
f2(v,1) = 1
f2(v,0) = 0

Somando todos os retornos:

10+6+3+1=2010 + 6 + 3 + 1 = 2010+6+3+1=20

Portanto, a saída exibida será:
20

d)

Em cada chamada recursiva da função f2, existe um laço que percorre os n primeiros elementos do vetor, portanto o custo de cada chamada é linear.

Além disso, a função chama a si mesma com n - 1, gerando a recorrência:

$T(n) = T(n-1) + n$

Com caso base:

$T(0) = \Theta(1)$

Expandindo a recorrência:

$T(n) = T(n-1) + n$
$= T(n-2) + (n-1) + n$
$= T(n-3) + (n-2) + (n-1) + n$ ....

Continuando até o caso base:

$T(n) = T(0) + 1 + 2 + 3 + \cdots + n$
Sabemos que:

$1+2+3+\cdots+n = n(n+1) / 2$
Portanto, a função cresce quadraticamente.

Logo, a complexidade assintótica total da função f2 é: $\mathbf{\Theta(n^2)}$

BOA PROVA !